GRAND CHALLENGE - 2025

Bitstream/MCS Generation and FPGA Target Configuration



DEVELOPED BY :

SHAKTI DEVELOPMENT TEAM @ IITM

SHAKTI.ORG.IN

Proprietary Notice

Copyright © 2025, **SHAKTI@IIT Madras**.All rights reserved.
Information in this document is provided “as is,” with every effort ensuring that the documentation is as accurate as possible.

SHAKTI@IIT Madras expressly disclaims all warranties, representations, and conditions of any kind, whether express or implied, including, but not limited to, the implied warranties or conditions of merchantability, fitness for a particular purpose and non-infringement.

SHAKTI@IIT Madras does not assume any liability rising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation indirect, incidental, special, exemplary, or consequential damages.

SHAKTI@IIT Madras reserves the right to make changes without further notice to any products herein.

The project was funded by Ministry of Electronics and Information Technology (MeitY), Government of India

Release Information

Version	Date	Updates	Reviewed by
1.0	15/09/2025	A Sree Satya Ananth	Kotteeswaran E
1.1	03/10/2025	A Sree Satya Ananth	Kotteeswaran E

Table of Contents

Proprietary Notice 2
Release Information 3
Table of Contents 3
1. Introduction 5
2. Software setup 5
3. Generating Bitstream and MCS Files for Target FPGA Boards 5
4. Pre-built Bitstream and MCS Files for Ganga and Yamuna SoCs 7
5. FPGA Configuration: Loading MCS and Bitstream Files via TCL Script 7
6. FPGA Configuration: Loading Bitstream via Vivado GUI 8
7. FPGA Configuration: Flashing MCS via Vivado GUI 10
8. FPGA Configuration: Status Check 14

1. Introduction

This document details the steps that need to be followed to

- generate bit stream
- Generate the MCS file.
- Program the bitstream
- Program the MCS file.

The ARTY-A7-100T board is considered as the target board.

2. Software setup

Ensure that all dependencies (such as the installation of required packages) mentioned in the document "GC2025 Tool Setup".

3. Generating Bitstream and MCS Files for Target FPGA Boards

The makefile, along with TCL scripts and a few YAML files, helps to create a terminal-based command that can be used to

- Configure the core, peripherals and memory.
- Compile the bsv files to generate Verilog files.
- Invokes the required IPs being used on the board.
- Generates the bitstream files and MCS files.
- Program the MCS to the target board.

The following steps can be used for this in a Ubuntu terminal.

Step 1: Clone the GC2025 repository from the Shakti Grand Challenge webpage by using the command "**git clone https://gitlab.com/shaktiproject/gc2025.git**".

Step 2: Change the directory to the cloned repository using the command "**cd gc2025/hw".**

Step 3: Activate the Python environment by running the command "**pyenv activate py(38 or above)".**

Step 4: Refer to the **README.md** file in the cloned repository for the correct instructions on generating the MCS and bitstream files for the target FPGA board.

For example, to generate the binary (bitstream) file and MCS file for the ARTY A7 board, run the following command:

"make build BOARD=arty_a7_yamuna XLEN_build=32" for Yamuna

or

"make build BOARD=arty_a7_ganga" for Ganga

The steps below will help to locate the path of the generated bitstream or MCS files for the target FPGA Board.

- The MCS file will be located at:
  gc2025/hw/fpga_project/c-class/fpga_top.mcs

- The bitstream (.bit) file will be located at:
  gc2025/hw/fpga_project/c-class/c-class.runs/core_impl_1/fpga_top.bit

```
## puts "DEBUG: ip_project_dir:    $ip_project_dir"
DEBUG: ip_project_dir:    ./fpga_project/manage_ip
## puts "DEBUG: core_project_dir:  $core_project_dir"
DEBUG: core_project_dir:  ./fpga_project/c-class
# set jobs [lindex $argv 0]
# open_project $core_project_dir/$core_project.xpr
Scanning sources...
Finished scanning sources
INFO: [IP_Flow 19-234] Refreshing IP repositories
INFO: [IP_Flow 19-1704] No user IP repositories specified
INFO: [IP_Flow 19-2313] Loaded Vivado IP repository '/scratch/vivado_18/Vivado/2018.3/data/ip'.
# set top_module [get_property TOP [get_filesets sources_1]]
# set bit $core_project_dir/$core_project.runs/core_impl_1/$top_module.bit
# set mcs $core_project_dir/$top_module.mcs
# set bitload "up 0x00000000 $bit"
# puts "BITSTREAM: $bit  MCS: $mcs"
BITSTREAM: ./fpga_project/c-class/c-class.runs/core_impl_1/fpga_top.bit  MCS: ./fpga_project/c-class/fpga_top.mcs
# write_cfgmem  -format mcs -size 16 -interface SPIx1 -loadbit $bitload  -force -file $mcs;
Command: write_cfgmem -format mcs -size 16 -interface SPIx1 -loadbit {up 0x00000000 ./fpga_project/c-class/c-class.runs/core_impl_1/fpga_top.bit} -force -file ./fpga_project/c-class/fpga_top
.mcs
Creating config memory files...
Creating bitstream load up from address 0x00000000
Loading bitfile ./fpga_project/c-class/c-class.runs/core_impl_1/fpga_top.bit
Writing file ./fpga_project/c-class/fpga_top.mcs
Writing log file ./fpga_project/c-class/fpga_top.prm
==================================
Configuration Memory information
==================================
File Format        MCS
Interface          SPIX1
Size               16M
Start Address      0x00000000
End Address        0x00FFFFFF

Addr1         Addr2         Date                    File(s)
0x00000000    0x003A607B    Oct  7 01:09:08 2025    ./fpga_project/c-class/c-class.runs/core_impl_1/fpga_top.bit
0 Infos, 0 Warnings, 0 Critical Warnings and 0 Errors encountered.
write_cfgmem completed successfully
# puts "MCS Generated !"
MCS Generated !
# exit
```

Figure 1.1: Bitstream and MCS files Generated Successfully

Figure 1.1 demonstrates the successful generation of both the bitstream file and the MCS file by executing the specified commands for either Ganga or Yamuna within the Ubuntu terminal.

Before configuring an FPGA, it's important to understand that configuration is the process of loading a design (bitstream) into the device to make it operational. The two main approaches are:

- **Bitstream configuration:** The design is directly downloaded (often via JTAG or similar interface) into the FPGA's volatile memory. This is temporary—the configuration is lost when power is removed and must be reloaded after each power cycle.

- **MCS (memory configuration storage) configuration:** The design (converted into an MCS file) is written to an external non-volatile memory such as SPI Flash. On power-up, the FPGA automatically loads the configuration from this memory, making the setup persistent across power cycles until the memory is rewritten.

First, will explore FPGA configuration using TCL scripts, followed by configuration using the Vivado GUI, both with bitstream and MCS files.

4. Pre-built Bitstream and MCS Files for Ganga and Yamuna SoCs

Pre-built .bit and .mcs files, ready-to-use working files for both Ganga and Yamuna SoCs, are available in the **"mcs folder"** located in the main branch of the GitHub repository. Specifically, these files can be found under the paths:

- **For Ganga:** gc2025/mcs/Ganga/ (containing the .bit and .mcs files)
- **For Yamuna:** gc2025/mcs/Yamuna/ (containing the .bit and .mcs files)

These files can be used as-is, without modifying the RTL or rebuilding the project, to run programs on the target FPGA boards. A detailed explanation of FPGA configuration will be provided in the next chapters.

5. FPGA Configuration: Loading MCS and Bitstream Files via TCL Script

After generating the bitstream (.bit) or MCS (.mcs) file for the target FPGA board, the following steps describe how to configure the FPGA using TCL scripts through the command line:

Step 1: Change Directory and Navigate to the hardware folder within the cloned repository by executing the command: **“cd gc2025/hw”**

This directory contains the necessary scripts and files needed for FPGA programming.

Step 2: Open and refer to the **“README.md”** file located in the **“hw”** folder. It contains detailed instructions and any board-specific considerations for configuring the FPGA using the MCS and Bitstream file via TCL script. This step ensures you follow the correct procedure for your specific FPGA board.

- Loading MCS File for the ARTY A7 board using the Ganga SoC, run:

  make program_mcs BOARD=arty_a7_ganga

- Loading MCS File for the ARTY A7 board using the Yamuna SoC, run:

  make program_mcs BOARD=arty_a7_yamuna

- Loading Bitstream File for the ARTY A7 board using the Ganga SoC, run:

  make program_bitstream BOARD=arty_a7_ganga

- Loading Bitstream File for the ARTY A7 board using the Yamuna SoC, run:

  make program_bitstream BOARD=arty_a7_yamuna

These commands trigger the TCL scripts that load the MCS file onto the FPGA, completing the configuration process.

6. FPGA Configuration: Loading Bitstream via Vivado GUI

Step 1:- Open Xilinx Vivado, go to the **“Tasks Tab”** and click on **“Open Hardware Manager”**.

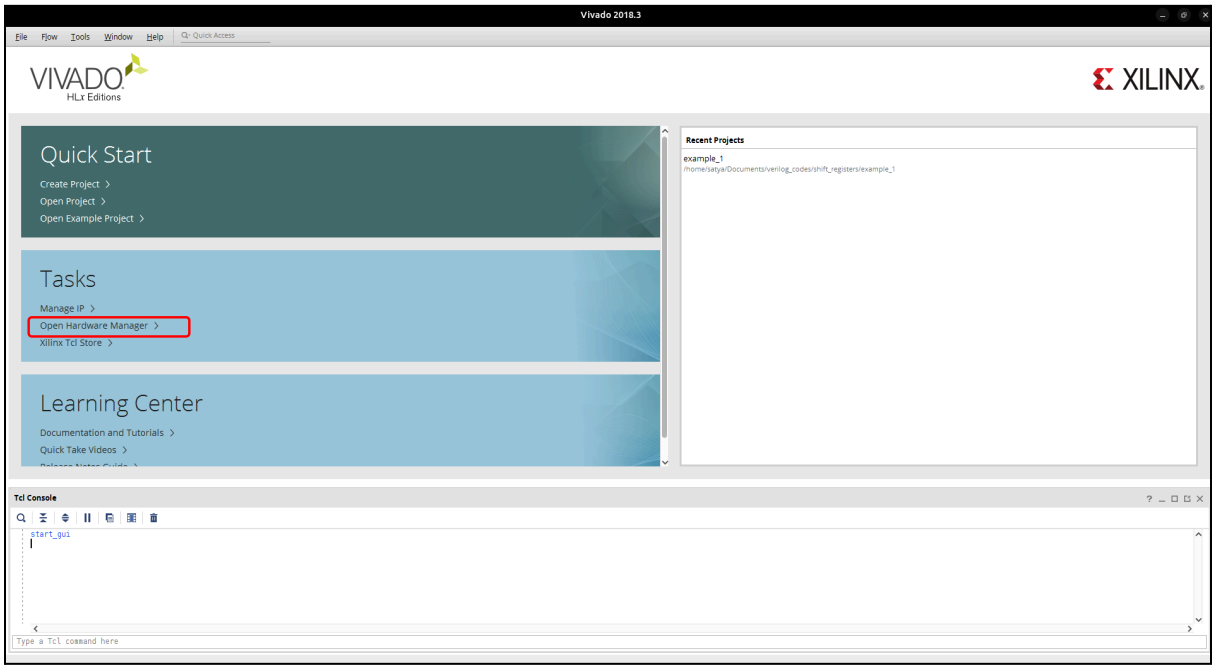


Figure 6.1: Vivado 2018.3

Step 2:- Click on **“Open Target”** and click on **“Auto Connect”**.

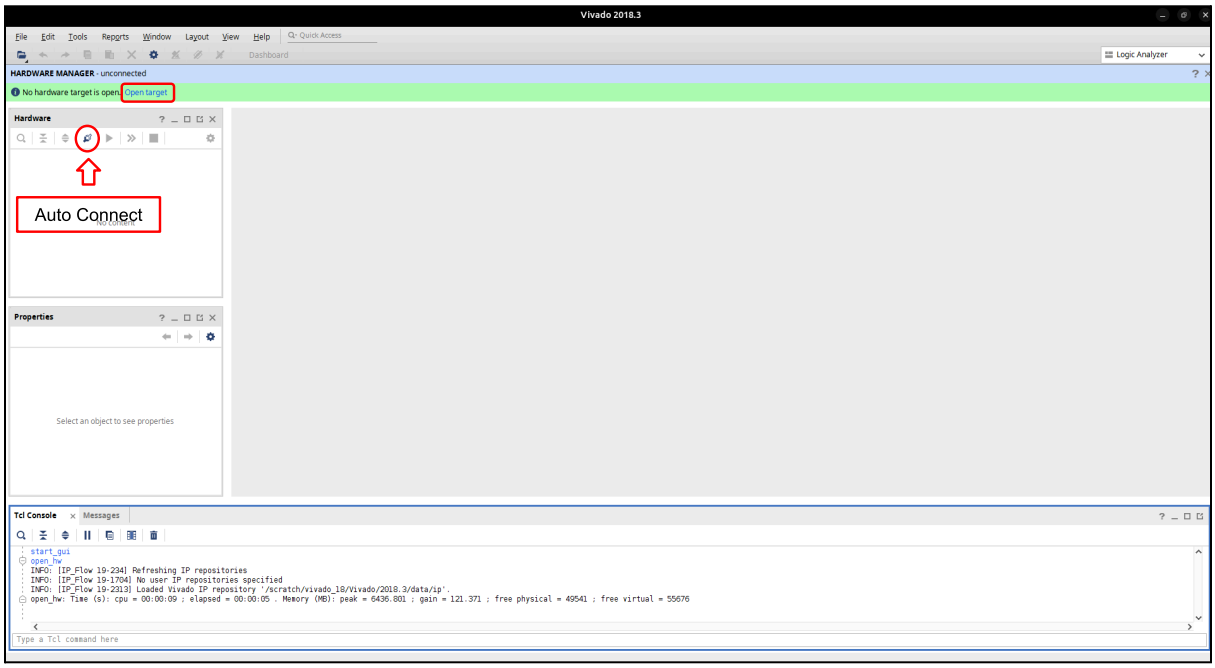


Figure 6.2: Open Target or Auto Connect

When the **ARTY_A7_100T** FPGA board is connected, the tool will display the **ARTY_A7_100T** part number as **xc7a100t_0(1)**. Please make sure the detection of the board by verifying the above part number. For other FPGA boards, the corresponding part numbers can be found online. Make sure the displayed part number matches the expected one for your board.

Step 3:- Select the **"Xc7a100t_0(1)"** under the name column, and make a right click on **"Programmed"** under the status column, and select **"Program Device".**

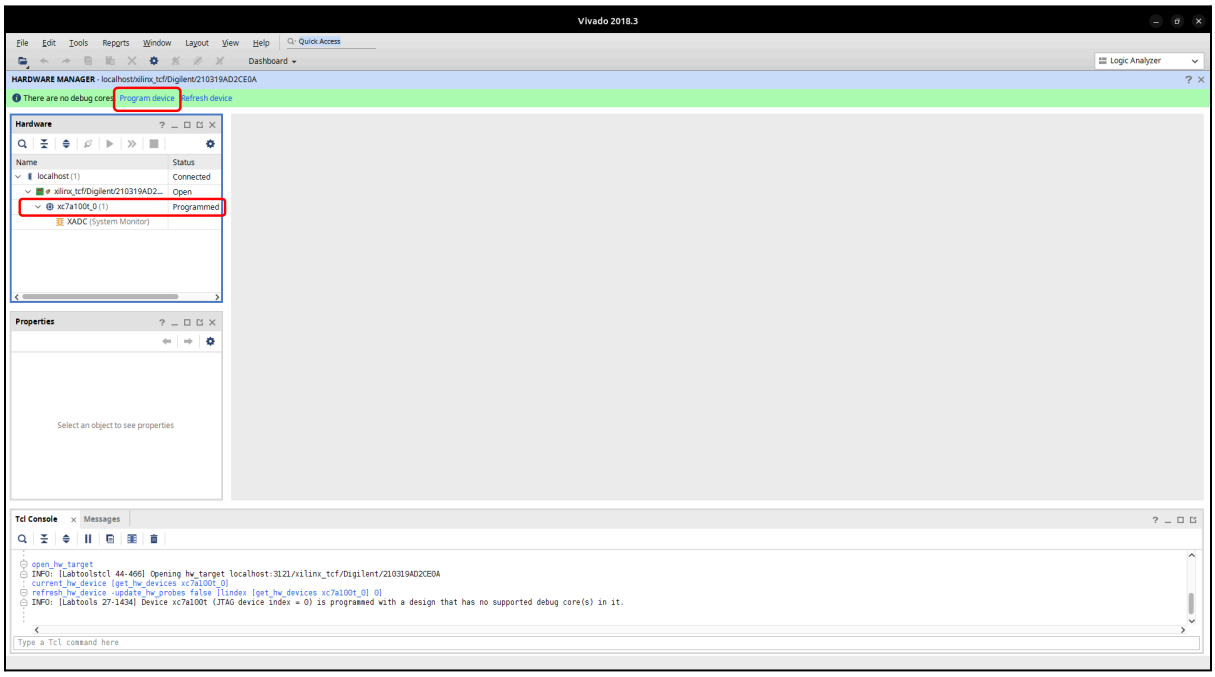


Figure 6.3: Selecting Target (Xc7a100t_0(1)) FPGA Board

Step 4:- Next, the "**Program Device**" Window will open. Select the bit stream file to be programmed in the bit stream file and click "Program".

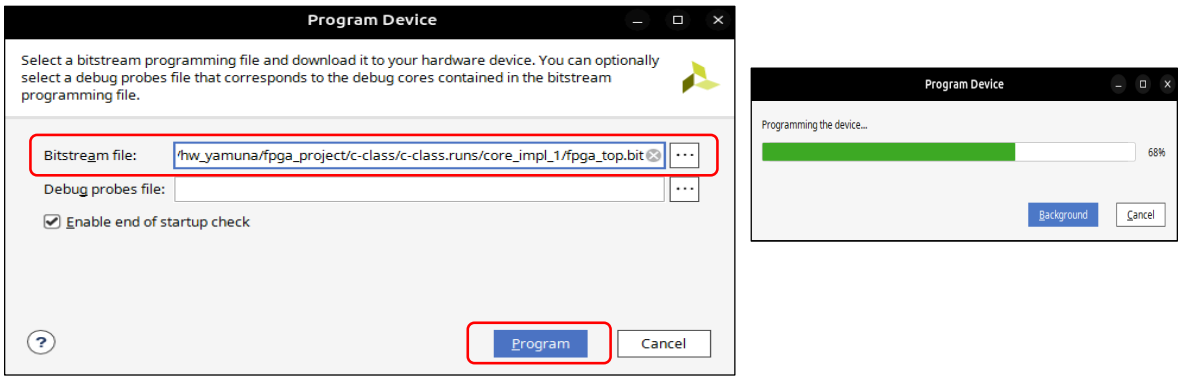


Figure 6.4: Programming the device through the .bit file, done successfully

Once the bitstream file is successfully programmed, the message **"Device xc7a100t is programmed"** will appear in the TCL console within the Vivado GUI. This message confirms that the bitstream file has been successfully loaded onto the **ARTY_A7_100T** FPGA board.

7. FPGA Configuration: Flashing MCS via Vivado GUI

Step 1:- Open Xilinx Vivado, go to the **“Tasks Tab”** and click on **“Open Hardware Manager”**.

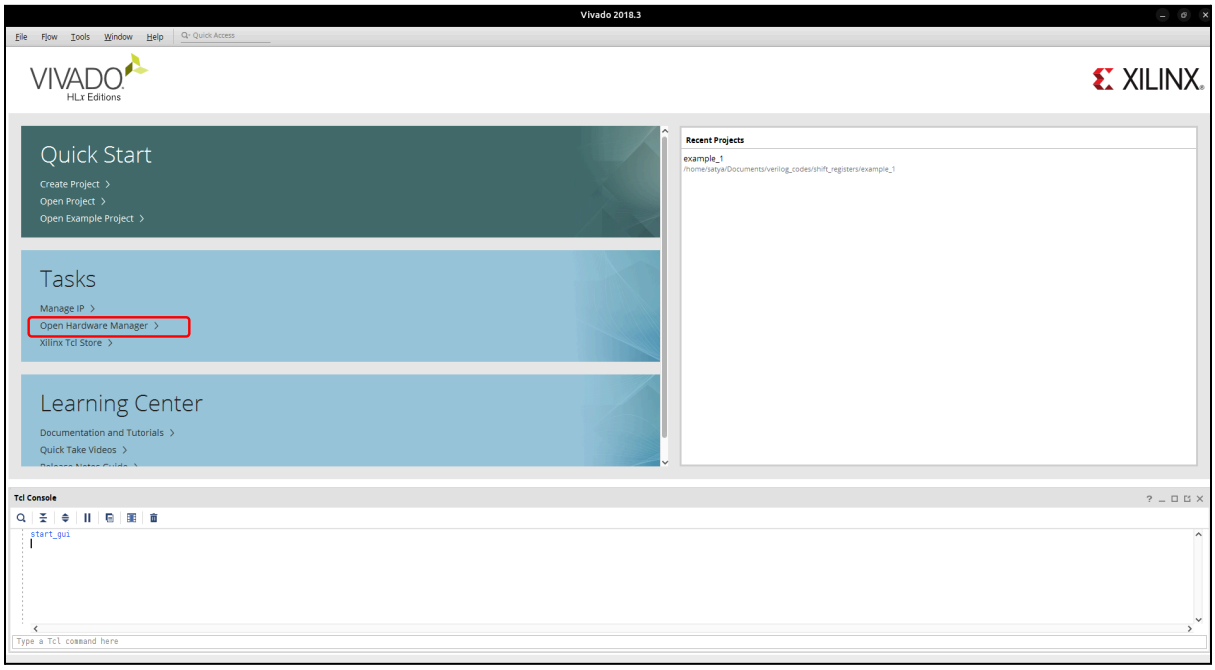


Figure 7.1: Vivado 2018.3

Step 2:- Click on **“Open Target”** or click on **“Auto Connect”**.

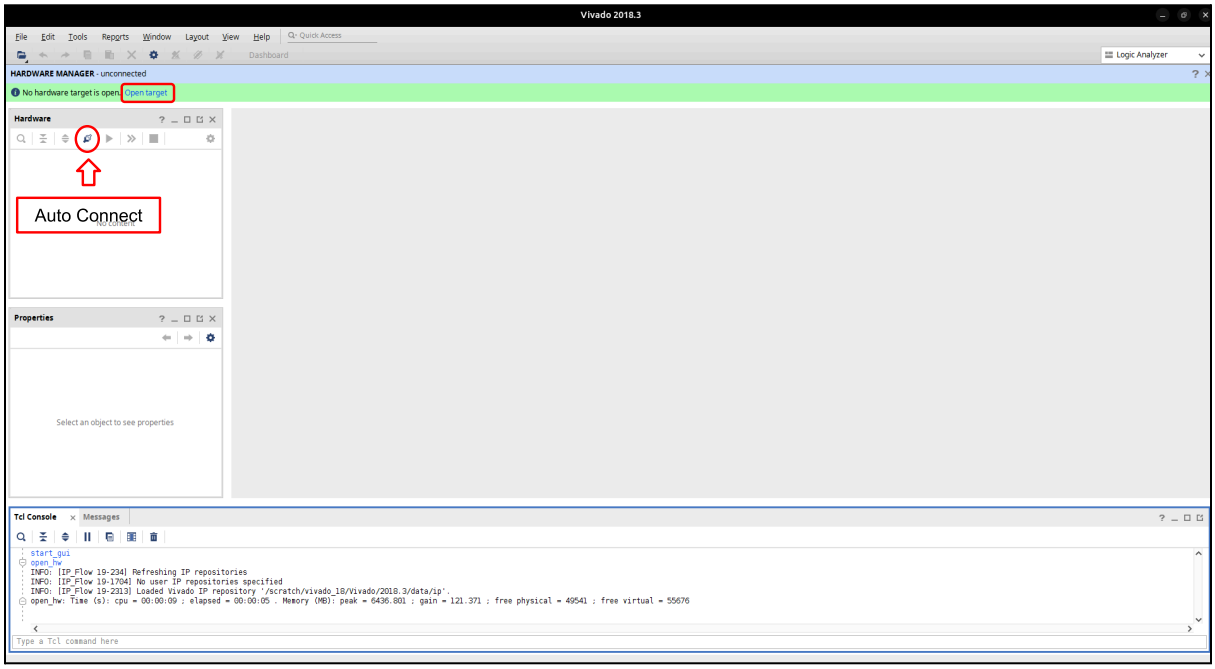


Figure 7.2: Open Target or Auto Connect

Step 3:- Select the **“Xc7a100t_0(1)”** under the name column, and make a right click on **“Programmed”** under the status column, and select **“Add Configuration Memory Device…”.**

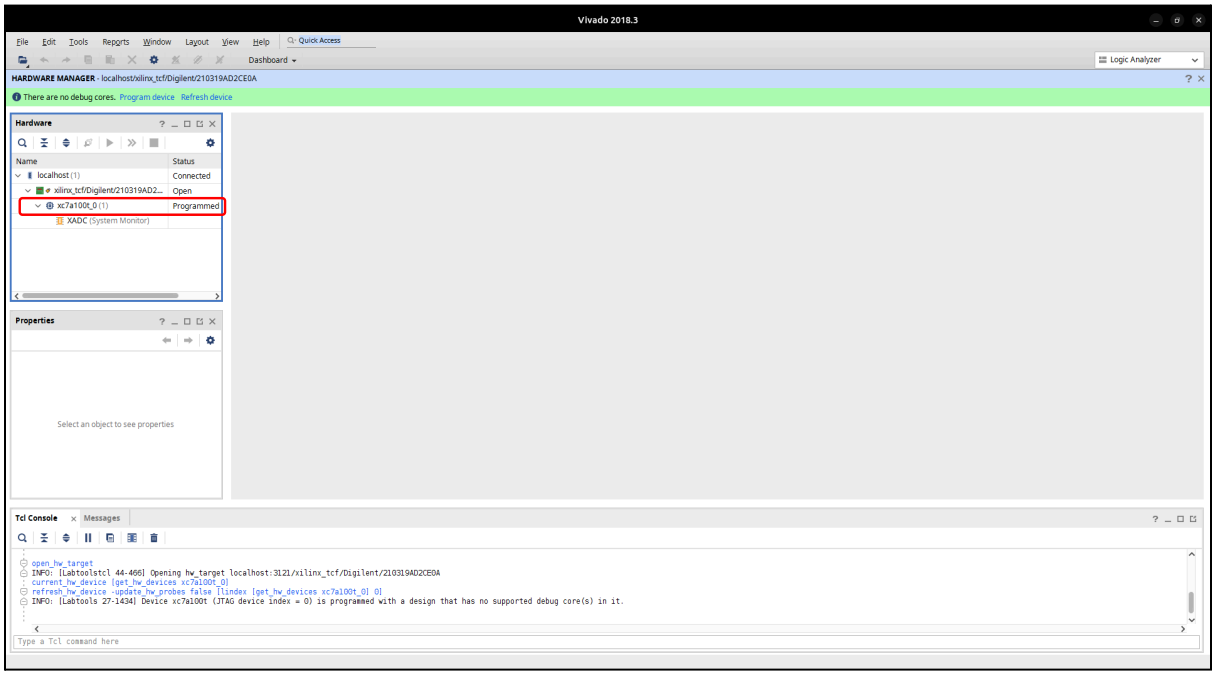


Figure 7.3: Selecting Target (Xc7a100t_0(1)) FPGA Board

Step 4:- In Filter, select **Manufacturer > Spansion** option, **Density (Mb) > 128**, **Type > SPI** & **Width > ALL,** and in Select **Configuration Memory Part** options select as shown below, Figure and click ok.

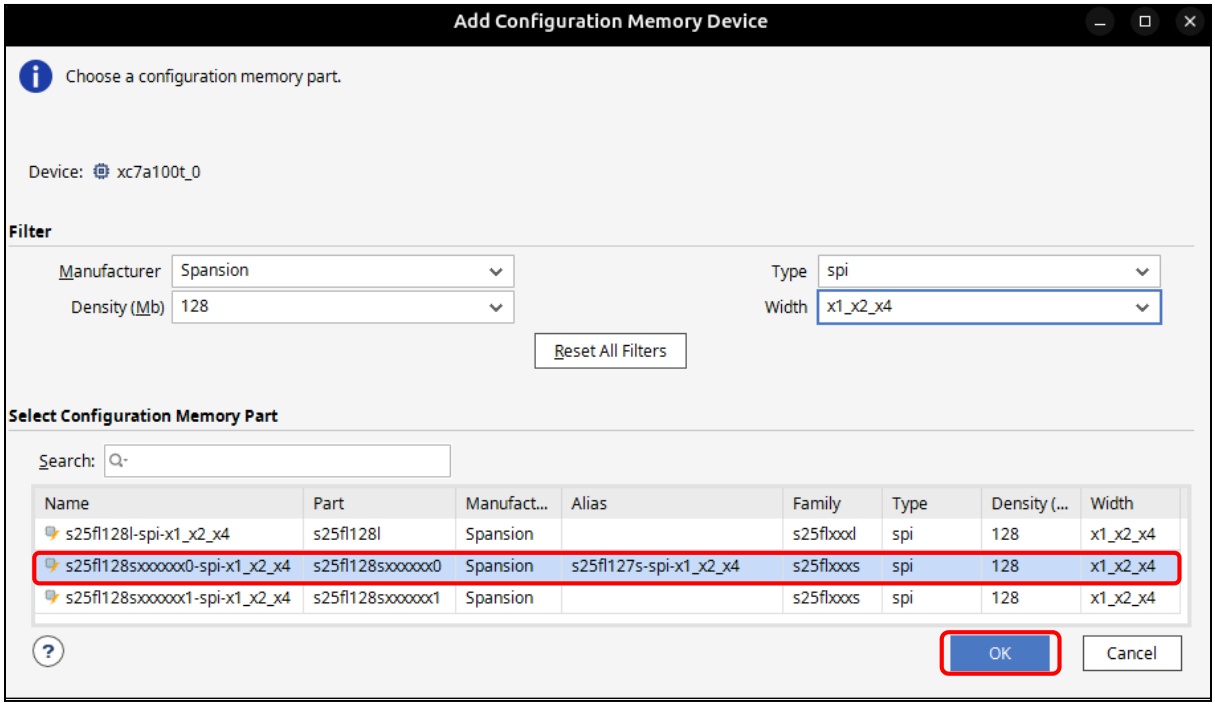


Figure 7.4: Adding a Configuration Memory Device

Step 5:- Next, after completion of the above step, another new dialogue box will appear as **"Add Configuration Memory Device Completed"**, as shown in the figure below, and click **"Ok"**.

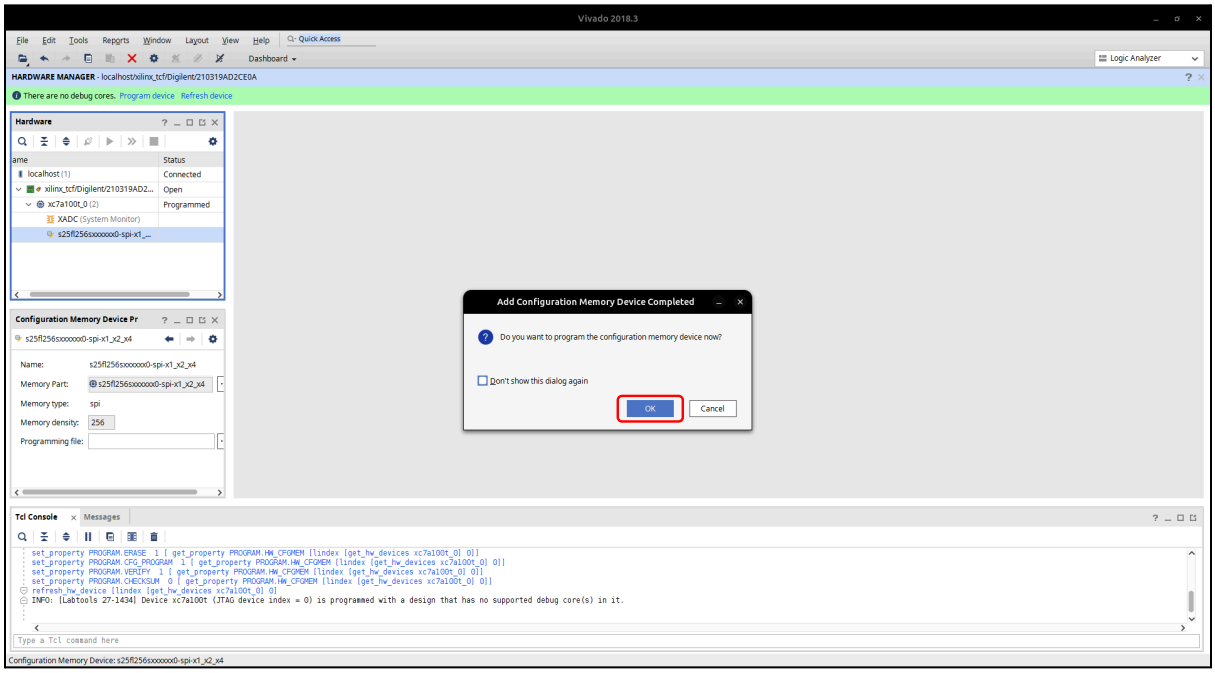


Figure 7.5: Successfully Added Configuration Memory Device

Step 6:- Next, the Program Configuration Memory Device Window will open. **"Add your .mcs file -> configuration file"**. In Program Operations, change the **"Address Range from Configuration file Only to Entire Configuration Memory Device"**. Click **"Apply"** and then **"OK"**.

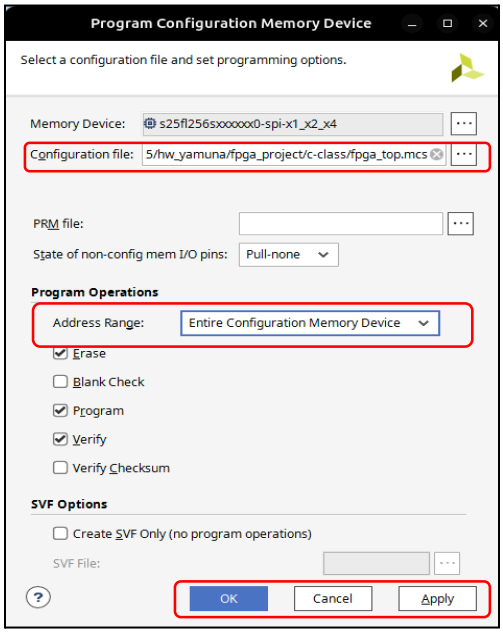


Figure 7.6: Program Configuration Memory Device

Step 7:- After clicking Ok, it will take ~2-3 minutes to complete the entire process of loading the .mcs file in your FPGA board, and it will be done in 2 steps as follows.

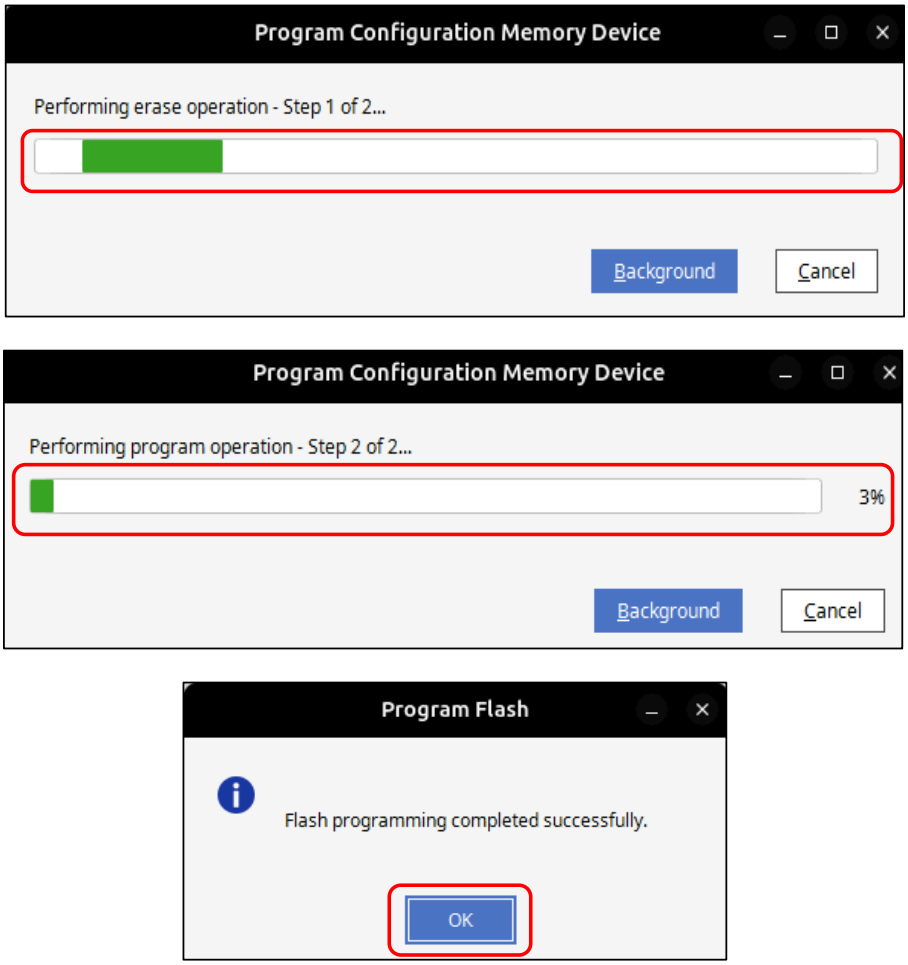


Figure 7.7: Programming the device through the .mcs file, done successfully

This indicates that the .mcs file has been successfully loaded onto the ARTY_A7_100T FPGA board. To confirm proper loading, either disconnect and reconnect your Arty board from your PC or press the Program button (as described in the FPGA Configuration: Status Check section). Then, connect OpenOCD; if the connection is successful, it confirms that the MCS configuration of the target FPGA board has been completed successfully.

8. FPGA Configuration: Status Check

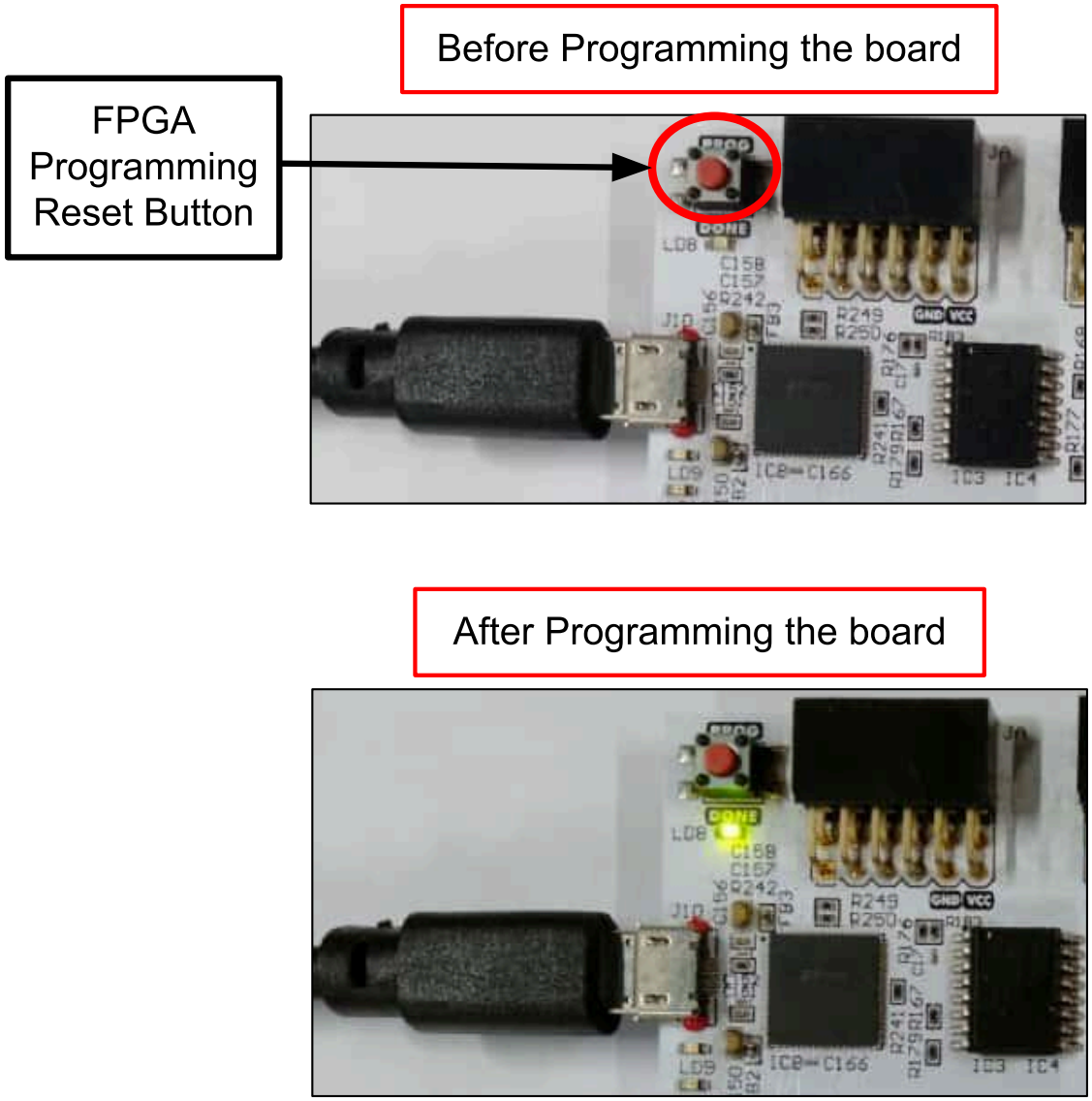


Figure 8.1: Before Programming vs After Programming